


# Midterm Project - Multimodal Sign Language Translation System

**Abdullah Hani Abdellatif Al-Shobaki - 2284612**

**Due To: 17 Dec 2025**

**Course: AIN4311 - Special Topics in Artificial Intelligence Engineering I**

**Instructor: Dr. Avşar Asan**

# Table of Contents


*1. Motivation ..... 3*

*2. System Overview ..... 3*

*Stage 1 — Vision ..... 3*

*Stage 2 — Text ..... 3*

*Stage 3 — Audio ..... 3*

*Pipeline Diagram ..... 3*

*3. Implementation Steps ..... 4*

*Step 1 — Data Collection ..... 4*

*Step 2 — Training the Classifier ..... 4*

*Step 3 — Sentence Generation ..... 5*

*Step 4 — Audio Output ..... 5*

*Step 5 — Real-Time System (run_live) ..... 5*

*4. Technical Information ..... 6*

*MediaPipe Hands ..... 6*

*MLP Classifier ..... 6*

*FLAN-T5-Large ..... 6*

*gTTS ..... 6*

*5. Problems Faced and Troubleshooting ..... 7*

*Problem 1 — MediaPipe did not install ..... 7*

*Problem 2 — NumPy binary error ..... 7*

*Problem 3 — OpenCV window freezing on macOS ..... 7*

*Problem 4 — Sentence generation failures ..... 7*

*Problem 5 — Overfitting risk ..... 7*

*6. Results ..... 7*

*Gesture Recognition ..... 7*

*Sentence Generation ..... 7*

*Audio Output ..... 8*

*7. Conclusion and Improvements ..... 8*

*Possible improvements ..... 8*

*8. Inputs Used ..... 8*

*9. Demonstration Video ..... 9*

# 1. Motivation

This project presents a **multimodal AI** solution that recognizes **sign language gestures** and translates them to English. The main goal is to understand and show how **Vision**, **Text**, and **Audio** can all connect inside one functional pipeline.

The system use a **computer vision** algorithm to detect the hand and a **machine-learning (ML) classifier** to recognize the gesture. Likewise, a language model creates an English sentence to be used afterwords with a text-to-speech model.

The motivation is both educational and practical:

- To learn multimodal AI practically in a project.
- AI has the potential to aid in communication and accessibility.
- To create a functioning system for the interaction of AI models.

# 2. System Overview

The system has **three** stages:

### Stage 1 — Vision

- A camera captures the user's hand.
- **MediaPipe Hands** extracts 21 hand landmarks (each has x, y, z).
- This creates a 63-value feature vector.

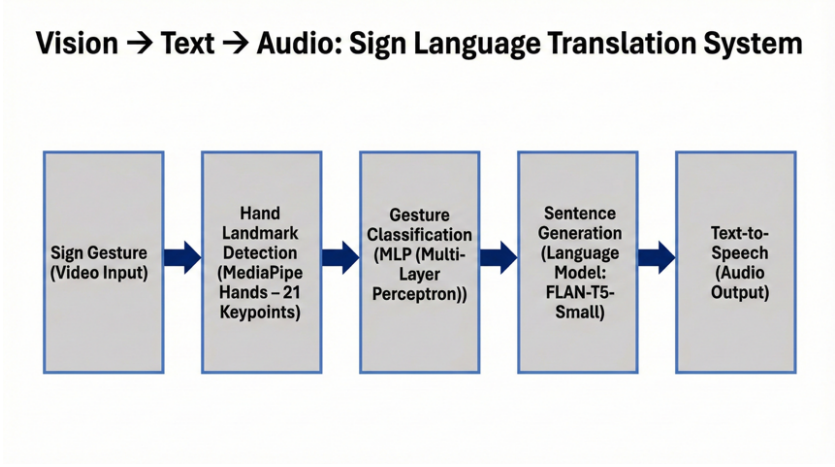
### Stage 2 — Text

- An **MLP classifier** predicts the gesture label.
- **FLAN-T5-Large** creates a short, natural English sentence that uses the gesture meaning.

### Stage 3 — Audio

- The **gTTS** model turns the sentence into an audio file.
- The MP3 file plays when clicked.

### Pipeline Diagram



# 3. Implementation Steps

## Step 1 — Data Collection

I collected 40 samples for each gesture using the webcam:

- **HELLO**
- **YES**
- **NO**
- **THANK YOU**

Every sample has 63 values of 21 landmarks of **MediaPipe**. The fucntion used for this step is **collect_data()**, which:

- Opens the webcam
- Detects the hand landmarks
- Saves the landmark values along with the label
- Shows progress made on the screen

```python
def collect_data(label, n=40):
    cap = cv2.VideoCapture(0)
    collected = []

    with mp_hands.Hands(max_num_hands=1) as hands:
        count = 0
        while count < n:
            ok, frame = cap.read()
            if not ok:
                break

            pts = get_landmarks(frame, hands)
            txt = f"{label}  {count}/{n}"
            cv2.putText(frame, txt, (10,30), cv2.FONT_HERSHEY_SIMPLEX, 1,(0,255,0),2)

            cv2.imshow("collect", frame)
            if cv2.waitKey(1) & 255 == ord('q'):
                break

            if pts is not None:
                collected.append(pts + [label])
                count += 1

    cap.release()
    cv2.destroyAllWindows()
    cv2.waitKey(1)
    return collected
```

## Step 2 — Training the Classifier

After collecting all samples, the data is saved in a CSV file.

I trained an **MLPClassifier** using **scikit-learn**. In which the important parts of this step:

- Input size = 63 features
- Hidden layers = (64, 32)
- Maximum iterations = 500
- Train/test split = 80/20

The **MLP** learns how the landmark patterns match each gesture.

```python
df = pd.read_csv("gesture_data.csv")
X = df.iloc[:,:-1].values
y = df.iloc[:,-1].values

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2)

mlp = MLPClassifier(hidden_layer_sizes=(64,32), max_iter=500)
mlp.fit(X_train, y_train)

preds = mlp.predict(X_test)
print("accuracy:", accuracy_score(y_test, preds))
```

## Step 3 — Sentence Generation

Initially, I used **FLAN-T5-Small**, but it was weak and inconsistent. Then I tried **FLAN-T5-Base**, which still made errors. I've finally brought **FLAN-T5-Large** successfully to produce stable, natural English sentences.

The **make_sentence()** function:

- Predicts gestures and responses such as **'hello'**, **'yes'**, **'no'**, **'thank you'**.
- Uses a custom prompt.
- Make a 6-12 word sentence in English that is natural.
- The gesture is verified with the meaning of the sentence.




## Step 4 — Audio Output

The **speak()** function converts text to speech using **gTTS**.

Steps performed:

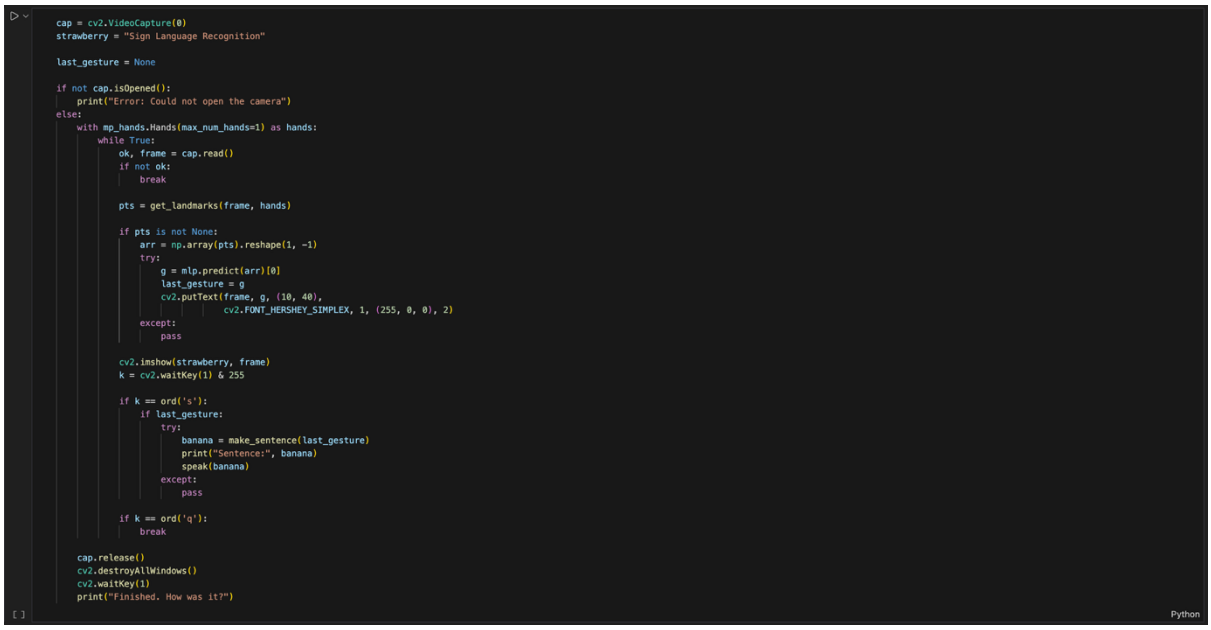
- Receives the English sentence generated.
- Creates an MP3 file.
- It plays when you click it inside notebook.




## Step 5 — Real-Time System (run_live)

The final system puts everything together, **run_live():**

- Opens the webcam
- Real-time detection of hand landmarks is done.
- Predicts the gesture
- Displays the result
- The program will generate and speak the sentence when pressing "s"
- Exits when pressing "q"



```
cap = cv2.VideoCapture(0)
strawberry = "Sign Language Recognition"

last_gesture = None

if not cap.isOpened():
    print("Error: Could not open the camera")
else:
    with mp_hands.Hands(max_num_hands=1) as hands:
        while True:
            ok, frame = cap.read()
            if not ok:
                break

            pts = get_landmarks(frame, hands)

            if pts is not None:
                arr = np.array(pts).reshape(1, -1)
                try:
                    g = mlp.predict(arr)[0]
                    last_gesture = g
                    cv2.putText(frame, g, (10, 40),
                                cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0, 0), 2)
                except:
                    pass

            cv2.imshow(strawberry, frame)
            k = cv2.waitKey(1) & 255

            if k == ord('s'):
                if last_gesture:
                    try:
                        banana = make_sentence(last_gesture)
                        print("Sentence:", banana)
                        speak(banana)
                    except:
                        pass

            if k == ord('q'):
                break

    cap.release()
    cv2.destroyAllWindows()
    cv2.waitKey(1)
    print("Finished. How was it?")
```

Python

# 4. Technical Information

## MediaPipe Hands

- Detects 21 hand landmarks
- Each landmark has coordinates of x, y, z.
- Creates 63-feature input vector
- It works well in real-time video.

## MLP Classifier

- The input layer size is 63, and the hidden layers are 64 → 32. The output has four gesture classes.
- Trained on 160 samples.

## FLAN-T5-Large

Reason for choosing large version:

- Best sentence accuracy
- Smaller versions ignored instructions
- The large model adheres to the rules of a prompt well.

## gTTS

- Takes English text
- Produces smooth audio
- Very simple to use
- Works well for real-time playback

## 5. Problems Faced and Troubleshooting

### Problem 1 — MediaPipe did not install

**MediaPipe** does not support **Python 3.13**. To solve such problem, I set up a new environment with **Python 3.10.11.**

### Problem 2 — NumPy binary error

Error: “dtype size changed”. To solve such problem, I install **numpy** again in the **3.10** environment.

### Problem 3 — OpenCV window freezing on macOS

Added **cv2.waitKey(1)** added to destroy window to avoid freezing

```
if cv2.waitKey(1) & 255 == ord('q'):
    break
```

### Problem 4 — Sentence generation failures

**FLAN-T5-Small** and **Base** produced bad sentences. To solve this problem, I used the **FLAN-T5-Large** model with a custom prompt.

### Problem 5 — Overfitting risk

The small size of the dataset (160 samples) was a problem and to prevent overfitting:

- Samples have been gathered from different places and lighting.
- Train/test split used
- Accuracy remained stable

## 6. Results

### Gesture Recognition

**MLP** model was able to identify all four gestures well with real-time prediction as well.

### Sentence Generation

**FLAN-T5-Large** generates stable, clear sentences such as:

- “I said yes to the idea.”
- “Thank you for your help today.”
- “The man said hello to the woman.”
- “I’m not going to do that.”

### Audio Output

**gTTS** produces:

- Clear voice
- Correct pronunciation
- Fast processing time

In general, the complete pipeline works correctly and gives the expected multimodal output.

## 7. Conclusion and Improvements

This project successfully displays integration of **Vision**, **Text**, and **Audio** in a single system. Gesture → Sentence → Speech works end-to-end.

### Possible improvements

- Add more gestures
- Collect more training samples
- Replace **MLP** out for a deep neural network
- Use a faster model utilizing a GPU.
- Add reverse translation (speech to sign)
- Create a more user-friendly user interface

## 8. Inputs Used

- There are a total of 160 gesture samples recorded with the webcam
- Four gesture classes
- Custom T5 prompt text
- Audio files generated through gTTS in MP3 format
- Full code in Jupyter Notebook .ipynb

## 9. Demonstration Video




**Note:** In case the video does not show in the PDF, please open the actual Word document of the report. Also, a copy of the demo video is included in the project folder **/images/demo_video/**.